Restricciones REST

- Restricciones REST
  - Restricciones de diseño REST
  - API RESTful
  - Modelo Cliente-Servidor
    - El Servidor
    - El Cliente
    - Intersección Cliente-Servidor
  - Restricciones
    - Sin estado
    - Cacheable
    - Interfaz Uniforme
    - Arquitectura en capas
    - Código bajo demanda

Restricciones de diseño REST

Para que una API sea considerada REST debe ajustarse a los siguientes principios
Estos principios también son conocidos como restricciones REST:

- Modelo Cliente-Servidor
- Sin Estado
- Cacheable
- Interface Uniforme
- Arquitectura en Capas
- Código Bajo Demanda **`(Opcional)`**

Algunos autores incluyen el principio HATEOAS como restricción, pero es mas una característica que se puede incluir en la respuesta:

- HATEOAS

API RESTful

El hecho de que una API funcione con el protocolo HTTP y use Json para transferir
la información no la hace una API REST.

Si una API cumple con todas las restricciones/principios arriba mencionados
se le denomina API RESTful.

Principios REST

Modelo Cliente-Servidor

Cliente y Servidor deberán ser dos entidades completamente separadas.
Esto permite que tanto servidor como cliente evolucionen independientemente uno del otro.

El Servidor

Será responsable de proporcionar un servicio y aceptar solicitudes referentes a ese servicio.
Este Servidor o Back-end se encargará del almacenamiento y procesamiento de los datos y solo debe entregarl el recurso solicitado.
No requiere saber que librería de interfaz de usuario implementa el cliente para desplegar los datos.

El Cliente

Necesita los servicios que proporciona el servidor.
Este Cliente o Front-end hará solicitudes al servidor y presentará la información.
La única información que debe conocer es la URL del recurso que solicita.
No necesita saber como se almacenan los datos, el modelo de datos usado o cual es la base de datos.

Intersección Cliente-Servidor

El punto de intersección entre el cliente y el servidor es el formato de los datos y su nivel de detalle

El formato de los datos
El servidor debe soportar el formato de dato que cliente requiera (Implementando módulos/librerías necesarios)
El cliente debe especificar en cada petición el formato que requiera.

Nivel de detalle de los datos
En una app de reserva de vuelos si el cliente necesita saber cuantos asientos hay disponibles, no le debemos
proporcionar información de las partes del avión. Se debe ser especifico para no desperdiciar ancho de banda.
De igual manera si el cliente solo quiere una foto de un album, no deberemos enviar todo el album.
Es decir, deberemos balancear la cantidad de información que proporciona cada endpoint y el número total de endpoints.

Principios REST

Restricciones

Sin estado

La especificación de nuestra API no debe asumir que se guarda el estado de peticiones anteriores.
El Cliente: Debe de dar al servidor toda la información necesaria para que el servidor pueda dar una respuesta.
El Servidor: No debe tener conocimiento de peticiones previas o guardar información (estado) acerca de ellas

Pros
Un sistema que no maneje estado es mucho mas fácil de implementar y de recuperar en caso de fallos.
El servidor al no tener que almacenar información entre solicitudes puede liberar recursos mas rápido.

Contras
Puede incrementarse el tráfico en la red debido a que tenemos que enviar mas información en cada solicitud.
En las respuestas sucede lo mismo podríamos estar enviando información que ya hayamos enviado previamente.

Ejemplo de una conversación (Stateful / Con estado)

- Dónde vive Juan Pérez? Nuevo Laredo
- Qué edad tiene? 40

Ejemplo de una conversación (Stateless / Sin estado)

- Dónde vive Juan Pérez?
- Qué edad tiene Juan Pérez?

Principios REST

Cacheable

Cache se refiere al almacenamiento temporal de información fuera del servidor, esto mejora enormemente el desempeño de la API.
Debido a la naturaleza sin estado de una API REST se incrementa el número de peticiones y la información que se transmite en ellas.
El cache es una manera de compensar esto. Cada solicitud debe especificar si puede ser almacenada en cache y por cuanto tiempo.

El servidor no puede conectar una serie de peticiones a un cliente en específico, cada petición es independiente.
Por ejemplo si en una petición deseamos saber la dirección del empleado y en otra petición queremos saber su
sueldo, el cliente deberá enviar el nombre o id del empleado siempre en cada petición.
Esta información del empleado podría almacenarse en un cache para que una futura petición no llegue al servidor
estando disponible para el cliente rápidamente ya que no es necesaria una solicitud al servidor.

El servidor debe especificar cuando esta información del cache debe ser refrescada para que la entidad que maneja el cache tome nota de esto.
Lo puede hacer mediante directivas de control de cache en los encabezados HTTP que determinan la "edad" máxima del cache,(Time-To-Live TTL)

Existen varios puntos entre el cliente y el servidor en donde existe cache:

- **Api Gateway:** Son como el mostrador de recepción en un edificio, enrutan llamadas y detienen visitantes inesperados.
- **Servidores Proxy:** Usualmente se colocan entre el cliente y el servidor y son compartidos, estos servidores pueden funcionar como cache.
- **En el Cliente/Aplicación/Dispositivo:** Este cache es privado y solo esta disponible para cada cliente de manera local.

Pros
Del lado del servidor se podrían ahorrar consultas a la BD.
Del lado del cliente se puede observar una mejora del desempeño.

Contras
Una desventaja del cache es que la información almacenada en el, podría quedar obsoleta.

Principios REST

Interfaz Uniforme

Diferentes y diversos tipos de clientes pueden interactuar con el servidor.
La especificación de la API establece un contrato entre los diferentes clientes y el servidor.

Los clientes necesitan la certeza de que pueden hacer una petición a una URL para obtener un dato.
Este dato estará en cierto formato que no va a cambiar. El que y el como obtendremos el dato de la API
deberá especificarse en su documentación.

Independientemente de donde vengan, todas las peticiones para un mismo recurso deben ser iguales
Estos recursos no deben ser muy grandes, pero deben contener todo lo que el cliente pueda necesitar.

Un endpoint en una API para consultar información de vuelos podría ser : **`/aeropuerto/vuelos/aerolínea/12345/`**
Y nos regresaría toda la información referente al vuelo **`12345`**
En que formato y que información obtendremos deberá estar especificado en la documentación de la API.
La cual a su vez deberíamos poder consultar en **`developer/aeropuerto`**

La información para desarrolladores en algunos sitios se puede consultar en **'developer.sitio.com'**
Ejemplo: **`walmart.com`** tiene **`developer.walmart.com`**

Pros
Se simplifica el trabajo entre los diferentes tipos de clientes al existir una única interface estándar.

Contras
El tener una interface estándar para todos los sistemas/plataformas podría ser una desventaja
ya que pudiera existir otra forma de comunicación más óptima o rápida en alguna de las plataformas.
pero estaremos restringidos a la interfaz de la API.

Principios REST

Arquitectura en capas

Esto nos permite simplificar un sistema complejo, separando sus diferentes componentes en capas. Estas capas se comunican entre sí.

Las API REST deben diseñarse para que ni el cliente ni el servidor puedan saber si se comunican con la aplicación final o con un intermediario.

Es decir el cliente no sabe si se esta comunicando con el servidor, el cache, o un API Gateway/Proxy.
El cliente solo debe saber el endpoint y no interactúa con la parte encargada de procesamiento ni con la base de datos.

Pros
En sistemas grandes y complejos esta separación en componentes independientes mas pequeños facilita el desarrollo, la escalabilidad y el mantenimiento.

Contras
Si nuestro sistema es pequeño esto puede agregar problemas de latencia, debido a la interacción entre las diferentes capas. Pudiéramos estar agregando complejidad a algo que se puede solucionar de manera mas simple.

Código bajo demanda

Esta es la única restricción que es opcional ya que algunos clientes podrán ejecutar código y otros no.
Generalmente las API REST envían recursos estáticos, pero la respuesta puede ser también
código ejecutable, como por ejemplo applets de Java o scripts de Javascript.

Comúnmente lo normal es que un cliente solo obtenga recursos, es decir información
de los endpoints de una API y procese esta información como sea necesario.

En el caso de un servidor Web y un cliente (navegador) web
este principio de código bajo demanda si es aplicable.

Principios REST